

CS3404 GUI Programming - Mini Project Report

AstraMart

Neon Commerce Galaxy

A cinematic Vue 3 SPA built with TypeScript, Tailwind CSS, Pinia, Vue Router, and DummyJSON products + authentication endpoints.

All bonus features included

Highlights

- Auth + JWT storage
- Persistent cart + favorites
- Dynamic product routes
- Manual dark mode toggle

Prepared for the CS3404 GUI Programming SPA challenge. AstraMart is intentionally designed to feel like a production-ready e-commerce dashboard rather than a standard CRUD demo.

Vue 3

TypeScript

Tailwind CSS

Pinia + Router

1. Implemented Features

The project satisfies every mandatory requirement from the brief and layers a stronger visual identity on top to target an A / A+ impression.

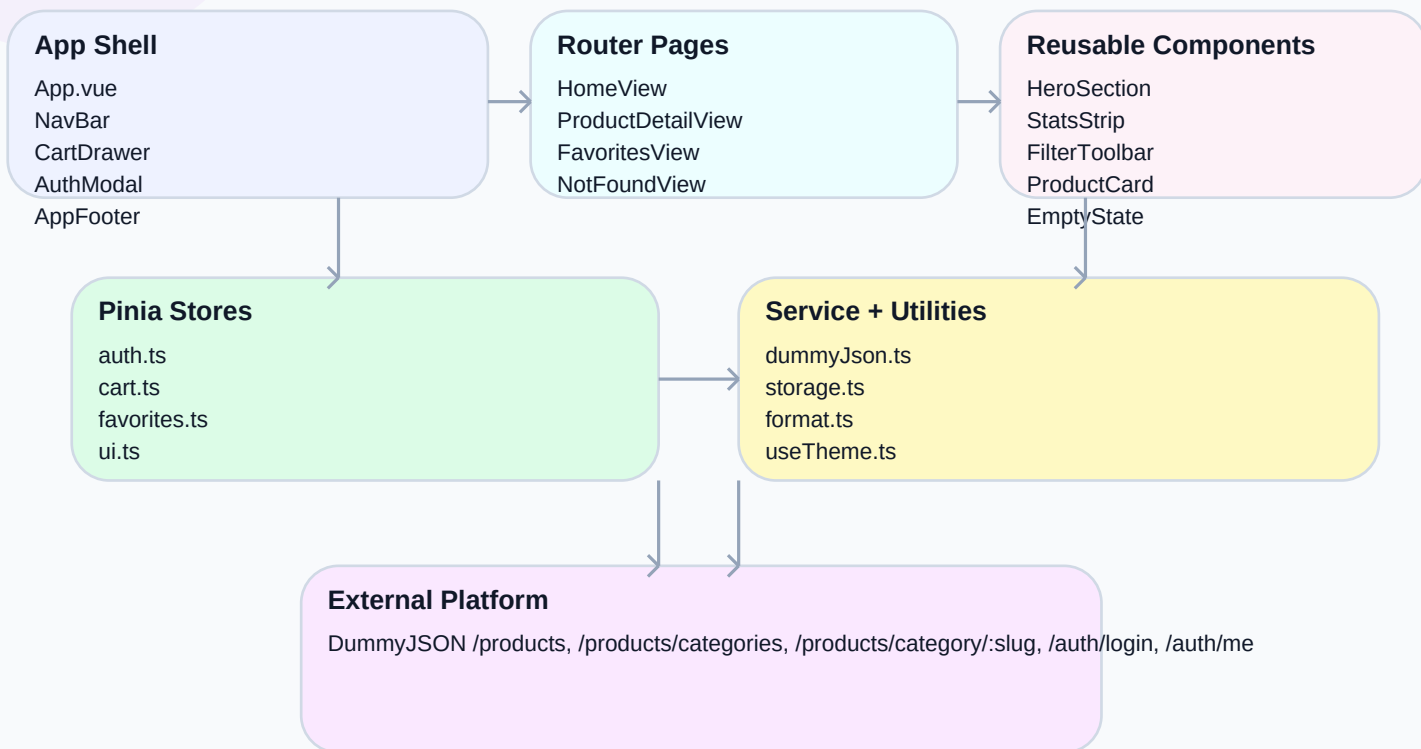
Category	What AstraMart includes	Why it matters
Mandatory	Data is fetched from DummyJSON product endpoints with strict interfaces and zero <i>any</i> usage.	Satisfies the typing and API-consumption requirements while keeping the service layer easy to explain.
Mandatory	Responsive grid, reusable cards, search input, category filters, sorting, and route-based detail pages.	Demonstrates props flow, component reuse, and modern SPA navigation behavior.
Bonus	Authentication simulation through <code>/auth/login</code> plus JWT token storage in <code>localStorage</code> .	Directly implements the login bonus feature with visible session state changes.
Bonus	Pinia-powered cart drawer and favorites page, both persisted across page reloads.	Shows global state management plus persistence using a realistic UX pattern.
Bonus	Manual dark mode toggle driven by Tailwind <code>dark:</code> classes and a class-based selector.	Adds visual polish and demonstrates theme-level responsiveness.
Design	Neon gradients, glassmorphism surfaces, animated ambient background, and a dashboard-style hero section.	Makes the mini project feel memorable and presentation-friendly for grading.

UI / UX design choices

- The hero section frames the app like a mission dashboard to make the landing screen visually stronger than a normal product list.
- Cards use a consistent rounded visual language, translucent panels, and brighter accent colors for hover and CTA states.
- The layout scales naturally from one-column mobile screens to a multi-column desktop grid with side-by-side detail pages.

2. Component Hierarchy and Architecture

The codebase is intentionally split into route pages, reusable presentation components, Pinia stores, and a thin API service layer. This keeps logic organized and avoids a single oversized App.vue file.



Architecture notes

- Route pages own data orchestration while smaller components focus on display and event emission.
- Stores handle cross-page state such as authentication, cart contents, favorites, and UI overlays.
- The DummyJSON service centralizes fetch logic and keeps request/response typing consistent across the app.
- Theme behavior is isolated in a composable so dark mode can be toggled from any UI layer without duplicating DOM logic.

3. State, API, and Deliverables Summary

AstraMart was planned to align closely with the required submission package while still feeling polished and creative.

API endpoints used /products?limit=0 /products/categories /products/category/:slug /products/:id /auth/login /auth/me	Global state auth store for session data cart store for purchase intent favorites store for saved items ui store for modal + drawer control
Persistence JWT tokens, cart items, favorite items, and selected theme are stored in localStorage so the interface survives page refreshes.	Deliverables included Complete Vue source code README.md with setup instructions Report.pdf with architecture explanation prompts.txt log in the required format

Final reflection

This project deliberately aims beyond a basic pass-level SPA. The combination of strict typing, reusable architecture, route-level detail pages, persistent state, and a visually memorable neon interface is intended to create the impression of a production-inspired front-end rather than a quick prototype.

Because the brief encourages students to prove they understand AI-assisted development, the code is organized so that each concern - UI composition, API access, global state, theme management, and formatting - is easy to explain during submission or viva-style questioning.